

Playwright Master Guide: Core Foundations & Assertions

Course: Rahul Shetty Academy - Playwright Automation

Topic: Framework Setup, Fixtures, CSS Selectors, and Web-First Assertions

1. Core CLI Commands

Keep this terminal cheatsheet handy for installation, maintenance, and test execution.

Command Syntax	Operational Purpose
<code>npm init playwright</code>	Installs the complete Playwright framework into your VS Code project directory.
<code>npm cache clean --force</code>	Purges the local npm cache to resolve broken dependencies or installation freezes.
<code>npx playwright test</code>	Executes all test suites sequentially inside terminal windows in headless mode.
<code>npx playwright test --headed</code>	Launches visible browser windows (headed mode) to visually observe execution.
<code>npx playwright show-report</code>	Starts a local web server to display a rich HTML report of the test results.
<code>npx playwright test --ui</code>	Launches Playwright's Interactive UI Mode for visual debugging, time-travel, and tracing.

2. Core Code Architecture & Fixtures

Understanding the Initialization Line

Every test file you write begins with this foundational import statement:

```
const { test, expect } = require('@playwright/test');
```

- `require('@playwright/test')`: Instructs Node.js to scan your `node_modules` directory, extract the Playwright dependency, and expose its libraries.
- `const { test, expect }`: Uses **Object Destructuring** to cleanly extract only the two core utilities you need:
 - `test`: The core block runner used to declare your test cases.
 - `expect`: The validation framework engine (assertion library).

Execution Modifiers

- **test.only**: Append **.only** to a specific test function to isolate execution. Playwright will execute **exclusively** that test and skip all others in the file.

```
test.only('Isolate this scenario', async ({ page }) => { ... });
```

Deep Dive: Fixtures Demystified

Playwright passes properties using a massive universal framework object. The global map order looks like this:

```
const testFixtures = {  
  browser: chromiumInstance, // Main browser engine instance  
  page: tabInstance,         // Specific individual browser tab  
  context: incognitoWindow,   // Isolated profile (incognito session)  
  request: apiTool,           // API testing utility  
  playwright: mainPackage    // Universal Playwright package reference  
};
```

Object Destructuring vs. Standard Arguments

You can capture fixtures in two distinct ways:

- **Method A: Without Destructuring (Passing Object Wrapper)**

```
test('My Test', async (fixtures) => {  
  // You must manually chain the object reference every single time  
  await fixtures.page.goto("https://google.com");  
});
```

- **Method B: Destructured (Recommended)** ✓

```
test('First Playwright test', async ({ page, context }) => {  
  // Direct access to variables inside the block  
  await page.goto("https://google.com");  
});
```

💡 **Why Destructuring with Curly Braces { } is Superior:** By wrapping variables inside curly braces, you tell JavaScript: *"Look inside this massive object, pluck out only the property matching this name, and assign it to a local variable."*

This enables **Flexible Ordering**. Because JavaScript looks up variables by property key name instead of index location, both configurations below execute identically:

- `async ({ page, context }) => {}`

- `async ({ context, page }) => {}`

3. CSS Selector Strategies

When you pass a locator string into `page.locator()`, Playwright automatically detects standard CSS rules. Below are the selector strategies ranked from basic to custom extensions:

1. Basic Selectors

- **Tag Name:** Identifies elements by their functional element node.

```
page.locator('button')
```

- **ID (#):** Targets unique element identification properties.

```
page.locator('#username')
```

- **Class (.):** Matches elements based on CSS visual styling group declarations.

```
page.locator('.btn-primary')
```

2. Attribute Selectors

- **Exact Match (=):** Matches strict text fields.

```
page.locator("input[type='email']")
```

- **Contains Substring (*=):** Matches dynamic elements containing specified partial phrases.

```
page.locator("input[placeholder*='Username']")
```

- **Starts With (^=):** Perfect for targeting IDs with changing suffixes (e.g., `user_123`, `user_456`).

```
page.locator("id^='user_'")
```

- **Ends With (\$=):** Captures fields matching a specific suffix phrase.

```
page.locator("input[name$='_submit']")
```

3. Structural & Child Relationships

- **Direct Child (>):** Grabs a child element exactly one nesting level below.

```
page.locator('form > input')
```

- **Descendant (Space):** Traverses deep nested layouts to grab matching targets anywhere inside a wrapper element.

```
page.locator('div.container input')
```

- **Next Sibling (+):** Selects an element located immediately following another sibling at the same level.

```
page.locator('label + input')
```

4. Pseudo-Classes (List Index Filtering)

- **First Child:** Selects the top element inside a matching collection container.

```
page.locator('ul.products li:first-child')
```

- **Last Child:** Selects the terminating element in a collection container.

```
page.locator('ul.products li:last-child')
```

- **Nth-Child:** Selects precise indexes. (Note: CSS index counters start at **1**).

```
page.locator('ul.products li:nth-child(3)') // Targets the 3rd element
```

5. Playwright Special CSS Extensions

- **Text Contents Matching (:has-text):** Isolates fields containing specific user-facing text strings.

```
page.locator("button:has-text('Log In')")
```

- **Internal Structural Containment (:has):** Targets parent containers exclusively if they contain a specified child item.

```
page.locator("div.card:has(button.delete-btn)")
```

4. Web-First Assertions vs. Immediate Checks

The Critical Architectural Difference

- **isVisible()** is an **Element Action Method**. It acts as an **Immediate Assertion** (Standard Boolean Check) which samples the DOM instantly at that exact millisecond. It must be paired with standard assertions like **expect(value).toBeTruthy()**.
- **toBeVisible()** is a native **Web-First Async Assertion** built directly into Playwright's testing validation framework.

Comparative Deep Dive

Feature	Web-First Assertion (toBeVisible())	Immediate Assertion (isVisible())
Type	Built-in Async Action Framework Validation	Standard JavaScript Boolean Property Check
Waiting Engine	Auto-waits (up to 5000ms) for elements to become stable	Zero waiting. Evaluates DOM instantly at that exact millisecond
Return State	JavaScript Promise (Resolves state or throws exception)	Pure synchronous boolean primitive (true / false)
Flakiness Risk	Extremely Low (safely handles slow connections & rendering)	Extremely High (prone to failing during UI animations)

 **Real-World Case Study (Why this matters for your tests):** Imagine clicking an add-to-cart button that takes **100 milliseconds** to render a success notification wrapper box.

- **isVisible() will fail ❌:** It checks the page state at 0ms. It detects an empty selector, returns **false**, and crashes the execution thread instantly.
- **toBeVisible() will pass ✅:** It checks at 0ms (not found), pauses execution safely, samples again at 100ms, detects the newly rendered notification, and seamlessly transitions to your next code line.

5. Web-First Assertion Cheatsheet

All methods below include automatic smart-retry wait routines to protect your automation framework scripts from unexpected timing failures.

A. Element State Assertions

```
await expect(locator).toBeVisible(); // Waits for element to appear on the UI
canvas
await expect(locator).toBeHidden(); // Waits for element to completely drop out
of DOM/view
await expect(locator).toBeEnabled(); // Waits for interactive buttons/inputs to
unlock
await expect(locator).toBeDisabled(); // Waits for elements to transition into an
unclickable state
```

B. Form and Input Assertions

```
await expect(locator).toBeChecked(); // Validates status changes for checkboxes
or radio options
await expect(locator).toBeEmpty(); // Confirms input fields or dynamic tables
contain no content
```

RegEx Mastery & GetBy Locators Topic: JavaScript Regular Expressions (RegEx) and User-Centric
getBy Locators



1. JavaScript Regular Expressions (RegEx) in Playwright

By default, Playwright searches text fields using flexible partial matches (substrings). Wrapping your target strings in **Regular Expressions** unlocks absolute control over text validation and dynamic data parsing.

Anatomy of an Exact-Match Pattern

Consider this strict validation pattern: `/^ZARA COAT 3$/i`

- `/` and `/`: Direct forward slashes act as delimiters. They instruct JavaScript that everything bound within them is a RegEx rule.
- `^`: The **Start-of-String** positional anchor. Forces evaluation to begin exactly at the first character.
- `ZARA COAT 3`: The literal sequence of text characters to search for.
- `$`: The **End-of-String** positional anchor. Forces evaluation to stop immediately after the final character (`3`).
- `i`: The case-insensitive execution flag. Evaluates `zara coat 3`, `Zara Coat 3`, and `ZARA COAT 3` as matching arguments.

Core RegEx Reference Blueprint

A. Global Modifiers (Flags)

Flags sit strictly after the trailing forward slash `/` to alter global matching engine behaviors.

- **`/pattern/i` (Case-Insensitive):** Ignores character casing discrepancies.
 - Example: `/zara/i` successfully hooks `"ZARA"`, `"Zara"`, or `"zara"`.
- **`/pattern/g` (Global Match):** Extracts or identifies all valid matches across a layout instead of ceasing operation at the first occurrence.

B. Positional Anchors (Strict Boundary Restrictions)

Anchors strip away default partial-matching behavior to validate strict element parameters.

- **`^` (Start of Line/String):** Text assets must begin directly with this pattern pattern.
- **`$` (End of Line/String):** Text assets must terminate directly with this pattern pattern.
 - Example: `/^Order \d+$/` successfully validates `"Order 123"`, but drops execution on `"Order 123 placed"`.

C. Character Classes (Data Type Matching)

Shorthand rules used to locate generic categories of characters.

- **`\d` (Digit):** Matches a solitary numerical digit parameter between `0-9`.
- **`\w` (Word Character):** Matches any single letter, number, or underscore string character.
- **`\s` (Whitespace):** Matches a lone space configuration, tab string indentation, or terminal newline command.
- **`.` (Wildcard):** Matches absolutely any singular character item (excluding native newline executions).

D. Quantifiers (Character Range and Constraints)

Quantifiers determine the precise repetition thresholds of the character or class preceding them.

- **`+` (One or More):** Iterates matches for one or more occurrences.
 - Example: `/\d+/` successfully groups variable number blocks like `"5"`, `"450"`, or `"9999"`.
- **`*` (Zero or More):** Iterates matches for zero or multiple occurrences.
- **`?` (Optional Flag):** Flags the preceding block as an optional item (zero or one instance).
 - Example: `/orders?/i` tracks match criteria for both single `"order"` and plural `"orders"` layouts.
- **`{n}` (Explicit Constraint):** Directs the engine to verify an exact number of occurrences.
 - Example: `/^\d{4}$/` maps criteria for sharp four-digit codes (like `"2026"`).

Practical Automation Case Studies

1. Tracking Variable String Numbers (Dynamic IDs)

For page elements that read `"Welcome, User #12345"` where the numerical identification tag fluctuates dynamically per run:

```
// Validates text starts with "Welcome, User #" and finishes with varying numeric lengths
```

```
await expect(page.locator('h1')).toHaveText(/Welcome, User #\d+/);
```

2. Asserting Against Dynamic Session URLs

For verification steps where targeted endpoints append variable transaction tokens or unique routing IDs:

```
// Confirms page URL explicitly starts with /cart/ and resolves cleanly on
// success
await expect(page).toHaveURL(/\/cart\/.*\/success$/);
```

⚠ Syntax Note: Inside standard regular expressions, raw forward slashes (/) found inside URL strings must be escaped using backslashes (\\) to prevent breaking the expression block boundaries.

3. Intercepting Alternating UI Elements (OR Logic)

For dynamic test assertions targeting action items or toggle states that change terminology randomly:

```
// The pipe symbol (|) sets up standard OR logic criteria
await page.getByRole('button', { name: /(Sign|Log) In/i }).click();
```

4. Handling Unpredictable Layout Paddings

```
// Decoupling an unexpected whitespace string wrapping targeted country text
// Logic: Line Starts -> 0 or more spaces -> Match 'India' -> 0 or more spaces ->
// Line Terminated
await expect(element).toHaveText(/^\\s*India\\s*$/i);
```

2. Playwright `getBy` Locators Cheatsheet

Playwright's native `getBy` selectors prioritize user-facing traits over structural DOM code. These methods emulate how a real user perceives, interacts with, and navigates an active application canvas.

1. `page.getByRole(role, options)` *Highly Recommended*

Targets items using native semantic **ARIA Accessibility Roles** across the browser tree. This strategy decouples scripts from fragile layout classes.

- **Target Elements:** `button`, `heading`, `checkbox`, `link`, `row`

```
<!-- Native App DOM -->
<button class="btn-primary">Submit Order</button>
```

```
// Automation Execution Script
await page.getByRole('button', { name: 'Submit Order' }).click();
```

2. `page.getByText(text, options)`

Locates UI assets based on raw string rendering. Employs flexible partial-matching strategies out of the box unless strict conditions are enabled.

```
<span>Order ID: 6a58ffce</span>
```

```
// Standard Search (Flexible substring verification)
await page.getByText('Order ID: 6a58ffce').click();

// Strict Verification (Enforces complete string precision match)
await page.getByText('6a58ffce', { exact: true }).click();
```

3. `page.getByLabel(text)`

Perfect for automated form parsing. It locates input nodes by querying their associated HTML structural `<label>` tags.

```
<label for="username">Enter Username:</label>
<input id="username" type="text" />
```

```
await page.getByLabel('Enter Username:').fill('myUser123');
```

4. `page.getByPlaceholder(text)`

Identifies editable form textboxes by tracking the gray contextual hint text shown to users prior to input.

```
<input type="email" placeholder="name@example.com" />
```

```
await page.getByPlaceholder('name@example.com').fill('test@test.com');
```

5. `page.getByTitle(text)`

Finds elements that leverage standard HTML tooltips or descriptive native properties.

```
<span title="More Information"> i </span>
```

```
await page.getByTitle('More Information').click();
```

6. `page.getByAltText(text)`

Targets graphics or visual content nodes by analyzing user-facing accessibility text descriptions (`alt` fields).

```

```

```
await page.getByAltText('User Profile Picture').click();
```

7. `page.getByTestId(id)`

A dedicated fallback strategy when user-facing visual text triggers are unavailable or highly unstable. Looks for `data-testid` properties injected by development teams.

```
<div data-testid="order-row-item">...</div>
```

```
await page.getByTestId('order-row-item').click();
```



3. Real-World Refactoring: Order History Table

The Legacy CSS Approach (Fragile & Structural)

```
// Relies heavily on structural raw tr indexes or generic bootstrap primary classes
await page.locator("li [routerlink*='myorders']").click();
```

The Modern User-Centric Approach (Resilient) ✓

This design isolates the parent table row containing your exact string match, then drops down to execute an action item using visible user-facing triggers.

```
// Step 1: Open orders page
await page.locator("li [routerlink*='myorders']").click();

// Step 2: Traverse rows, isolate by dynamic tracking value, target operational
// control element
await page.locator("tr")
  .filter({ hasText: cleanOrderId })
  .getByRole('button', { name: 'View Order' })
  .click();
```

Why this approach scales frameworks:

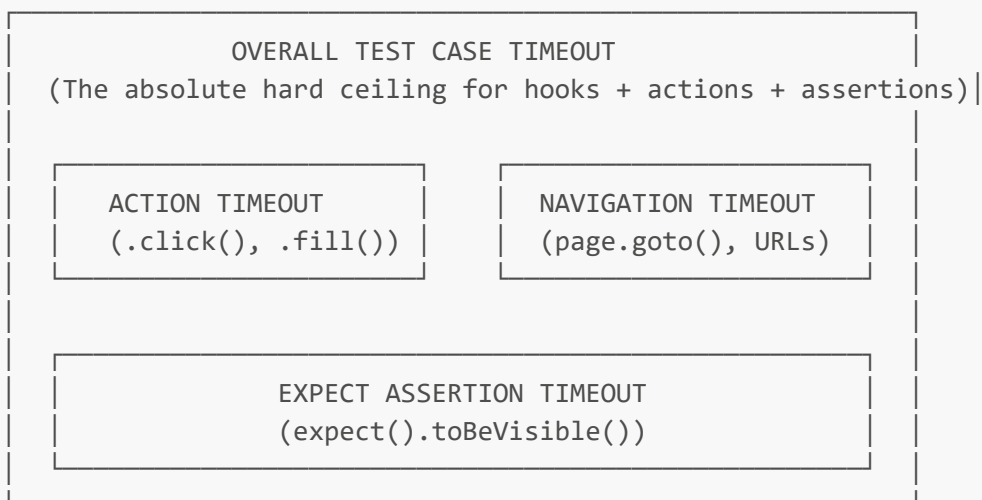
- **Mirrors User Journeys:** Automation actions correspond exactly with how manual QA engineers or consumers read a web page.
- **Resilient to Structural Redesigns:** If frontend developers swap a framework template class (e.g. changing `class="btn-primary"` to a utility style like `class="bg-blue-500 rounded p-2"`), your `getByRole` script passes without requiring updates.

The Timeout Hierarchy

Topic: Comprehensive Execution Timeouts, Cascade Rules, and Configuration Overrides

Overview of the Timeout Architecture

Playwright breaks execution delays into four distinct, independent layers. Understanding this hierarchy allows you to troubleshoot hanging tests or slow network pipelines without masking flakiness.



1. Expect Assertions (Web-First Assertions)

Controls how long a web-first async assertion (e.g., `expect(locator).to...`) will actively poll the DOM waiting for a specified state condition to pass.

A. Global Level Override (`playwright.config.js`)

Configured at the root configuration object level.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  expect: {
    timeout: 40 * 1000, // 40 seconds default limit applied across all assertions
  },
});
```

B. Test/Suite Level Override (`expect.configure`)

Generates an isolated, pre-configured validation engine instance for localized use inside a specific file, suite, or code block.

```
const slowExpect = expect.configure({ timeout: 9000 });

// This assertion leverages the 9000ms threshold override
await slowExpect(page.locator(".my-4").first()).toHaveText("Shop");
```

C. Step Level Override (Inline Argument)

Overrides all higher configurations exclusively for that specific validation line.

```
await expect(page.getByText("Success!")).toBeVisible({ timeout: 6000 });
```

Resolution Hierarchy Cascade

1. **Step Level Override** 
2. **Test/Suite Level Override** (`expect.configure`) 
3. **Global Level Config** (`expect.timeout`) 
4. **Framework Default** (Stands at exactly **5,000ms** if completely omitted globally)

2. Action Timeouts

Controls how long an individual user-emulation action method (such as `.click()`, `.fill()`, or `.hover()`) will wait for target web elements to satisfy actionability prerequisites (visibility, stability, and interactivity checks).

A. Global Level Override (`playwright.config.js`)

Defined nested squarely inside the shared operational `use` context object block.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    actionTimeout: 10 * 1000, // 10 seconds default for all actions
  },
});
```

B. Step Level Override (Inline Argument)

Overrides the standard global action metrics exclusively for the current interactive command.

```
await page.getByRole("link", { name: "Shop" }).click({ timeout: 15000 });
```

Resolution Hierarchy Cascade

1. **Step Level Override** →
2. **Global Level Config** (`use.actionTimeout`) →
3. **Framework Default** (Evaluates to 0 / **Infinite Wait**, restricted solely by the Overall Test Case Timeout limit)

3. Navigation Timeouts

Governs the maximum execution window allocated for network routing, client-side lifecycle loads, and redirect triggers (such as `page.goto()`, `page.waitForURL()`, or click-driven page changes).

A. Global Level Override (`playwright.config.js`)

Configured cleanly inside the shared web operational `use` context container block.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    navigationTimeout: 30 * 1000, // 30 seconds default constraint for web page routing
  },
});
```

B. Step Level Override (Inline Argument)

Alters network connection waiting behavior strictly for that targeted navigation action.

```
await page.goto('https://example.com', { timeout: 15000 });
```

📋 Resolution Hierarchy Cascade

1. **Step Level Override** ➡
2. **Global Level Config** (`use.navigationTimeout`) ➡
3. **Global Level Action Config** (`use.actionTimeout`) ➡
4. **Framework Default** (Evaluates to `0` / **Infinite Wait**, restricted solely by the Overall Test Case Timeout limit)

🚨 4. Overall Test Case Timeout (The Hard Ceiling)

This represents the ultimate macro execution ceiling boundary. It enforces a strict expiration timer covering the total lifespan of an individual test block execution lifecycle—encompassing test hooks (`beforeEach`, `afterEach`), text entries, validations, and teardowns.

A. Global Level Override (`playwright.config.js`)

Configured at the root structural layer of your workspace parameters.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  timeout: 40 * 1000, // Imposes a hard stop limit at exactly 40 seconds per
  single test case
});
```

B. Test Case Level Override (`test.setTimeout`)

Extends or shrinks the structural timeline bounds directly within an operational test context block.

```
test('Execute complex slow flow scenario', async ({ page }) => {
  // Dynamically opens the processing ceiling allowance to 60 seconds for this
  single run
  test.setTimeout(60000);

  await page.goto('https://rahulshettyacademy.com');
  // ... Additional test instructions
});
```